

# SKILL WEEK 9

## S17: Question 1

### Aim

To implement a simple job-management system that can create background jobs, assign Job IDs, maintain their Process IDs (PIDs) and states, list active jobs, display status information, update job status when processes finish, and test job listings.

### Program:

```
sindhuj@SINDHUJA: ~/skill_ x + v
GNU nano 7.2 skill9_task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 5

struct Job
{
    int jobID;
    pid_t pid;
    char state[20];
};

int main()
{
    struct Job jobs[MAX_JOBS];

    for (int i = 0; i < MAX_JOBS; i++)
    {
        pid_t pid = fork();

        if (pid == 0)
        {
            sleep(3 + i);
            exit(0);
        }
        else
        {
            jobs[i].jobID = i + 1;
            jobs[i].pid = pid;
            strcpy(jobs[i].state, "Running");
        }
    }

    printf("\nACTIVE JOB LIST\n");
    printf("-----\n");
    printf("Job ID\tPID\tStatus\n");

    for (int i = 0; i < MAX_JOBS; i++)
    {
        printf("[%d]\t%d\t%s\n",
            jobs[i].jobID,
            jobs[i].pid,
            jobs[i].state);
    }
}
```

|                |                     |                    |                 |                   |                      |
|----------------|---------------------|--------------------|-----------------|-------------------|----------------------|
| <b>^G</b> Help | <b>^O</b> Write Out | <b>^W</b> Where Is | <b>^K</b> Cut   | <b>^T</b> Execute | <b>^C</b> Location   |
| <b>^X</b> Exit | <b>^R</b> Read File | <b>^L</b> Replace  | <b>^U</b> Paste | <b>^J</b> Justify | <b>^_</b> Go To Line |

```
sindhuja@SINDHUJA: ~/skill_ x + v
GNU nano 7.2 skill9_task1.c *
    }
    else
    {
        jobs[i].jobID = i + 1;
        jobs[i].pid = pid;
        strcpy(jobs[i].state, "Running");
    }
}

printf("\nACTIVE JOB LIST\n");
printf("-----\n");
printf("Job ID\tPID\tStatus\n");

for (int i = 0; i < MAX_JOBS; i++)
{
    printf("[%d]\t%d\t%s\n",
           jobs[i].jobID,
           jobs[i].pid,
           jobs[i].state);
}

printf("\nUpdating Job Status...\n");

for (int i = 0; i < MAX_JOBS; i++)
{
    waitpid(jobs[i].pid, NULL, 0);
    strcpy(jobs[i].state, "Done");
}

printf("\nFINAL JOB LIST\n");
printf("-----\n");
printf("Job ID\tPID\tStatus\n");

for (int i = 0; i < MAX_JOBS; i++)
{
    printf("[%d]\t%d\t%s\n",
           jobs[i].jobID,
           jobs[i].pid,
           jobs[i].state);
}

return 0;
}
|

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify
^C Location  ^_/ Go To Line
```

Output:

```
sindhuja@SINDHUJA: ~/skill_ x + v
sindhuja@SINDHUJA:~$ mkdir skill_week9
sindhuja@SINDHUJA:~$ cd skill_week9
sindhuja@SINDHUJA:~/skill_week9$ nano skill9_task1.c
sindhuja@SINDHUJA:~/skill_week9$ gcc skill9_task1.c -o skill9_task1
sindhuja@SINDHUJA:~/skill_week9$ ./skill9_task1

ACTIVE JOB LIST
-----
Job ID  PID    Status
[1]     638    Running
[2]     639    Running
[3]     640    Running
[4]     641    Running
[5]     642    Running

Updating Job Status...

FINAL JOB LIST
-----
Job ID  PID    Status
[1]     638    Done
[2]     639    Done
[3]     640    Done
[4]     641    Done
[5]     642    Done
sindhuja@SINDHUJA:~/skill_week9$
```

**Observation:**

1. Five background jobs were created using fork().
2. Each job received a unique Job ID and PID.
3. Initially, all jobs were displayed with the status Running.
4. After waitpid() completed, the status of each job changed to Done.

**Result:**

The job-management program successfully listed active jobs, displayed Job IDs and PIDs, updated job status information, and showed the final job list after all background jobs completed.

S17: Question 2

**Aim:**

To identify a target job, transfer terminal control, resume stopped processes, wait for completion, update job state, and test foreground switching using Linux job control commands.

```
sindhuja@SINDHUJA:~/skill_week9$ sleep 300 &
[1] 721
sindhuja@SINDHUJA:~/skill_week9$ sleep 350 &
[2] 722
sindhuja@SINDHUJA:~/skill_week9$ sleep 400 &
[3] 723
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]      721  Running                  sleep 300 &
[2]-    722  Running                  sleep 350 &
[3]+    723  Running                  sleep 400 &
sindhuja@SINDHUJA:~/skill_week9$ fg %2
sleep 350
^Z
[2]+  Stopped                  sleep 350
sindhuja@SINDHUJA:~/skill_week9$ |
```

The fg %2 command transferred terminal control from the shell to Job 2, making it the foreground process. The sleep 350 process started running in the foreground.

**Result:**

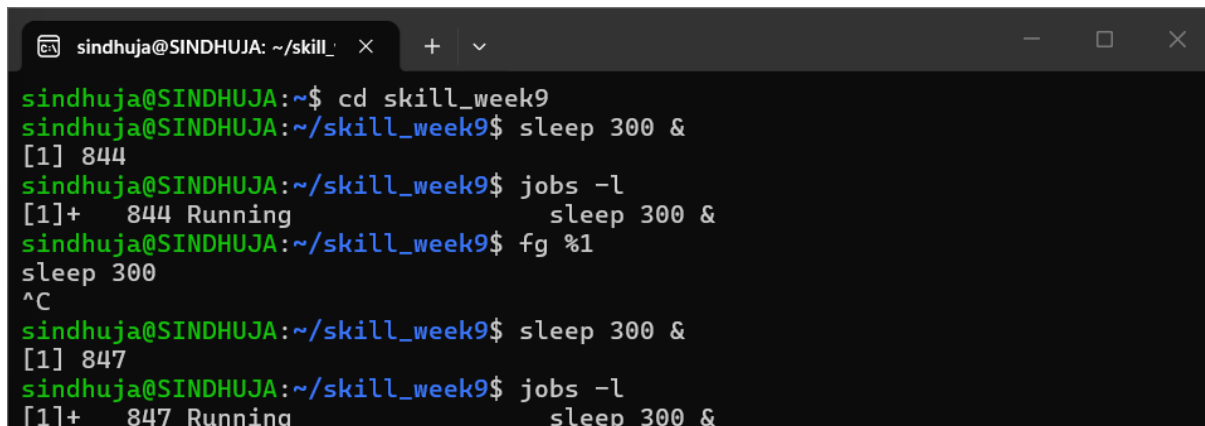
The Linux job control commands were successfully implemented to identify a target job, transfer terminal control from the background to the foreground using fg, stop the foreground process using Ctrl + Z, resume it in the background using bg, update the job state, and verify foreground switching successfully

S18: Question 1

## Aim:

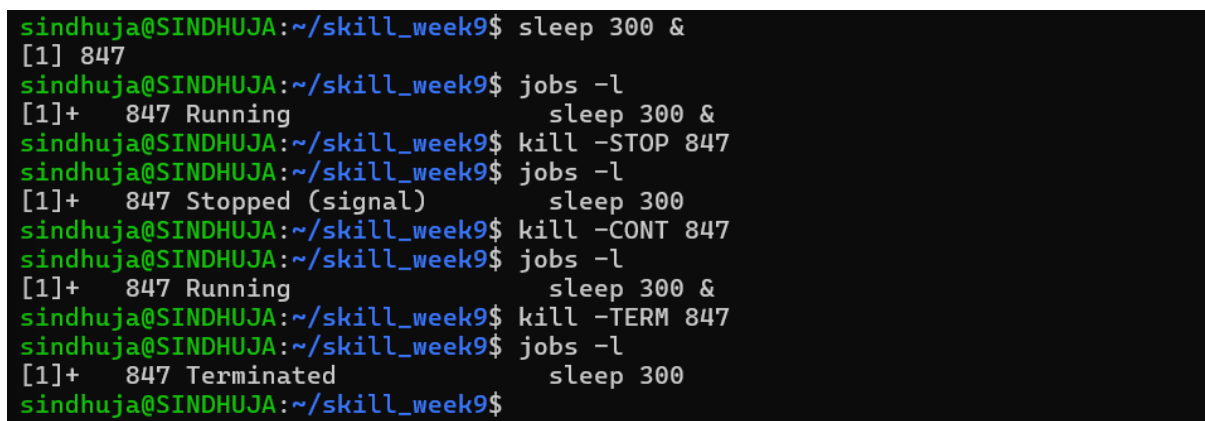
To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior using Linux signal commands.

## Test SIGINT:

A terminal window titled 'sindhuja@SINDHUJA: ~/skill\_...' showing a series of commands and their outputs. The user navigates to the 'skill\_week9' directory and runs 'sleep 300 &' in the background, which returns PID 844. They then use 'jobs -l' to see '[1]+ 844 Running sleep 300 &'. Pressing Ctrl+C (SIGINT) sends a signal to the foreground process, which then returns to the prompt. The user then runs 'sleep 300 &' again, returning PID 847, and 'jobs -l' shows '[1]+ 847 Running sleep 300 &'.

```
sindhuja@SINDHUJA:~$ cd skill_week9
sindhuja@SINDHUJA:~/skill_week9$ sleep 300 &
[1] 844
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]+  844 Running                  sleep 300 &
sindhuja@SINDHUJA:~/skill_week9$ fg %1
sleep 300
^C
sindhuja@SINDHUJA:~/skill_week9$ sleep 300 &
[1] 847
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]+  847 Running                  sleep 300 &
```

## Test SIGSTOP, SIGCONT, SIGTERM:

A terminal window showing the testing of SIGSTOP, SIGCONT, and SIGTERM. The user runs 'sleep 300 &' (PID 847) and 'jobs -l' shows it as running. They then use 'kill -STOP 847', and 'jobs -l' shows '[1]+ 847 Stopped (signal) sleep 300'. Next, they run 'kill -CONT 847', and 'jobs -l' shows '[1]+ 847 Running sleep 300 &'. Finally, they run 'kill -TERM 847', and 'jobs -l' shows '[1]+ 847 Terminated sleep 300'.

```
sindhuja@SINDHUJA:~/skill_week9$ sleep 300 &
[1] 847
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]+  847 Running                  sleep 300 &
sindhuja@SINDHUJA:~/skill_week9$ kill -STOP 847
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]+  847 Stopped (signal)         sleep 300
sindhuja@SINDHUJA:~/skill_week9$ kill -CONT 847
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]+  847 Running                  sleep 300 &
sindhuja@SINDHUJA:~/skill_week9$ kill -TERM 847
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]+  847 Terminated             sleep 300
sindhuja@SINDHUJA:~/skill_week9$
```

## Observation:

1. Signal delivery was tested using SIGINT, SIGSTOP, SIGCONT, and SIGTERM.
2. SIGINT interrupted the foreground process.
3. SIGSTOP paused the background process.
4. SIGCONT resumed the stopped process.
5. SIGTERM terminated the running process.

## Result :

Signal delivery and terminal signal behavior were successfully analyzed using Linux signal commands. The process responded correctly to interrupt, stop, continue, and terminate signals.

## S18: Question2

### Aim

To implement background job recovery in Linux by resuming stopped jobs using continue signals, updating job status, maintaining the job table, verifying process execution, and testing background recovery using Linux job control commands.

```
sindhuja@SINDHUJA: ~/skill_
sindhuja@SINDHUJA:~$ cd skill_week9
sindhuja@SINDHUJA:~/skill_week9$ sleep 300 &
[1] 884
sindhuja@SINDHUJA:~/skill_week9$ sleep 350 &
[2] 885
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]- 884 Running                  sleep 300 &
[2]+ 885 Running                  sleep 350 &
sindhuja@SINDHUJA:~/skill_week9$ kill -STOP 885
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]- 884 Running                  sleep 300 &
[2]+ 885 Stopped (signal)         sleep 350
sindhuja@SINDHUJA:~/skill_week9$ kill -CONT 885
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]- 884 Running                  sleep 300 &
[2]+ 885 Running                  sleep 350 &
sindhuja@SINDHUJA:~/skill_week9$ kill -STOP 885

[2]+  Stopped                    sleep 350
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]- 884 Running                  sleep 300 &
[2]+ 885 Stopped (signal)         sleep 350
sindhuja@SINDHUJA:~/skill_week9$ bg %2
[2]+ sleep 350 &
sindhuja@SINDHUJA:~/skill_week9$ jobs -l
[1]- 884 Running                  sleep 300 &
[2]+ 885 Running                  sleep 350 &
sindhuja@SINDHUJA:~/skill_week9$ jobs
[1]- Running                     sleep 300 &
[2]+ Running                     sleep 350 &
sindhuja@SINDHUJA:~/skill_week9$ fg %2
sleep 350

[1]- Done                        sleep 300
sindhuja@SINDHUJA:~/skill_week9$
sindhuja@SINDHUJA:~/skill_week9$ jobs
sindhuja@SINDHUJA:~/skill_week9$
```

### Observation:

1. Two background jobs were created successfully.

2. The selected job was stopped using the SIGSTOP signal.
3. The stopped job was resumed using SIGCONT.
4. The bg command resumed the stopped job in the background.
5. The jobs command verified the status of running and stopped jobs.
6. After bringing the job to the foreground with fg, all jobs completed and were removed from the job table.

**Result:**

Successfully resumed stopped jobs, updated job status, maintained the job table, verified execution, and tested background recovery using Linux job control commands.